

Simrule: AI-Assisted Review of Auto Insurance Underwriting Decline Rules

<YOUR FULL NAME> | AI Prototyping for Business Innovation (Ivey Online, delivered on Uplimit), Weeks 4-5
Innovation Challenge | October 5, 2026

Live app: <https://jevwithwind.github.io/simrule/> **Code:** <https://github.com/jevwithwind/simrule>

Regulatory Assistant chatbot: [link added after Chatbase setup] **Avatar video:** [link added after recording]

Academic prototype for Ivey Online coursework. Not affiliated with or representative of the Financial Services Regulatory Authority of Ontario (FSRA). All insurer submissions and precedent records are fictional.

1. Executive Summary

Ontario's Take All Comers regime lets an auto insurer decline a consumer only on grounds the regulator has approved, so each proposed decline rule decides who can buy a product the law makes compulsory. Reviewing these filings is document-heavy and judgment-heavy, and similar rules from different insurers can receive different answers. The challenge was to show how AI could make that review faster and more consistent while strengthening transparency, fair consumer outcomes and human accountability.

Simrule is a working prototype of that review workbench, live as a static web app. Five connected parts cover the brief: a **document upload interface** that extracts rules from PDF, Word or text filings and asks the reviewer to confirm them; a **compliance dashboard** for a queue of 100 fictional submissions; a **flagging system** that records findings against 15 criteria in the framework's four Levels, each with issue codes, verbatim evidence and citations; a **recommendation panel** with similar rules, a consumer lens and next steps; and a **decision workflow** in which reviewers accept or override every finding, divergence triggers senior sign-off, and the reviewer must confirm ownership of the rationale. A chatbot pack, an architecture diagram and an avatar script share the same vocabulary.

The central design choice is that the AI never writes the recommendation. It produces structured findings; fixed, published scoring code turns them into an advisory recommendation; a person decides. All 100 test rules have a recommendation (21 approve with conditions, 22 request more information, 57 recommend decline), reasoning against every criterion, explained similar rules and next steps. The potential impact is a review that starts from structured evidence rather than a blank page, applies one standard to every filing and leaves a complete audit trail, while every outcome remains a human decision.

2. Methodology

Each component was built with the tool that best fitted its job:

Component	Tool	Why
Web app (all five features)	Claude Code building React, TypeScript and Vite; GitHub Pages	Used instead of Bolt, Lovable or Replit because the project needed a data pipeline, tests and version control in one repository, not only screens. No backend; state stays in the browser.
100 assessments	Claude as a batch assessment analyst, versioned prompt, schema validator	Precomputing gives every rule a full, reviewable assessment and lets the same checks run on all of them.
Similar rules	TF-IDF plus structured feature overlap	Transparent and fast in a browser; embeddings are the production path.
Chatbot	Chatbase: 10 source files, 28 Q&A pairs	Quick to configure; shares the knowledge base and vocabulary of the app.

Component	Tool	Why
Avatar and diagram	HeyGen script; hand-laid SVG exported to PNG and PDF	An executive summary under three minutes and a one-page view of where humans stay in control.

What worked best

- **Schema-first prompting.** Every assessment had to pass a schema and content validator: all 15 criteria, 2 to 4 sentences of reasoning, a question for the insurer and two next steps. The validator, not my impression, decided when the prompt was good enough.
- **An issue-code taxonomy.** The brief's 21 codes plus four I added (prohibited factor, less restrictive alternative, subjective judgment, privacy intrusion) gave findings one vocabulary that the dashboard, consistency monitor and chatbot reuse.
- **Judgment separated from scoring.** Level 1 is a gate; Levels 2 to 4 are weighted 40, 35 and 25; thresholds are 75 and 50. All 15 weight and threshold variants matched the 13 anchor rules, so the starting values were frozen rather than tuned.
- **Verbatim evidence and a verified-citation registry.** All 940 evidence quotes are tested as exact substrings, so the app can highlight them. Official legal sites were blocked from the build environment, so the 21 legal references are held at instrument level and marked "section not verified".

How I iterated

The first prompt (v1) was piloted on 10 rules and only 1 passed validation: the recommendations already pointed the right way, but reasoning shrank to one-word findings and evidence was missing. A critical review mapped 13 problems to 13 new prompt rules (v2); the re-run passed 10 of 10 with no recommendation changing. In the batch run, 9 of the other 90 assessments failed first-pass validation, all for single-sentence reasoning. Similarity also needed a second version: broad trigger categories caused ties and false matches until I split them into 36 trigger types. A consistency audit then found 29 missing "pending similar submission" flags, 9 explanations that never said why similar rules had different outcomes, and 2 misapplied findings; 38 fixes across 29 assessments cleared them without changing any recommendation. A 15-rule spot check agreed with 12, partly with 1 and disagreed with 2, which I left as override candidates rather than editing findings to fit. Finally, 134 unit tests, 6 end-to-end tests and an accessibility scan found 4 bugs that type checks missed, including PDF upload failing in Chromium.

3. Limitations and Risks

Simrule's limits are deliberate and published on its How it works page, but each matters for a real deployment.

What production needs that the prototype lacks

- **Live, server-side AI.** The 100 assessments were generated once and frozen. New uploads get low-confidence keyword triage unless a visitor runs the optional live assessment with their own API key. Production needs a server-side model with retrieval over a versioned regulatory corpus.
- **Authentication, roles and secure storage.** Roles are a demonstration switch and data lives in the browser. Confidential filings need access control, encryption and records retention.
- **Integration and validation.** Filings should arrive from the regulator's systems, not manual upload, and the scoring weights must be back-tested against historical decisions, not only 13 anchor rules I chose.

- **Service and monitoring.** French-language service, accessibility testing with real reviewers, and ongoing monitoring of model quality, cost and latency.

Risks before deployment

- **Hallucinated or stale citations.** Citations are restricted to an id-based registry with a verified flag, but none of the 21 legal references has yet been checked against the official text (listed in VERIFY.md).
- **Automation bias and rubber-stamping.** A reviewer who accepts everything adds nothing. Simrule requires a decision on all 15 findings and shows the acceptance rate, treating very high agreement as a warning sign.
- **Proxy discrimination the model misses.** Language, postal code or benefit status can track protected grounds. Keyword triage catches some wording; subtle proxies need reviewers and disparate-impact testing.
- **Gaming.** Insurers could learn the issue codes and draft rules that avoid flags while keeping the same effect, so reviewers must judge substance and the taxonomy must evolve.
- **Model and regulation drift; over-reliance on a precomputed set.** Each assessment is stamped with prompt and framework versions so it can be re-run when guidance changes, and the consistency monitor tracks overrides by issue code to reveal drift.
- **Privacy.** Filings can contain personal information. The optional live assessment sends text to an external API and must stay off for confidential data.

How human oversight is built in

People act at every step that changes an outcome. The reviewer confirms each extraction before triage, then accepts or overrides all 15 findings, with a reason code and note for every override; the score recomputes and the change is marked as human. Senior sign-off is required when the decision differs from the AI, a Level 1 finding is overridden, confidence is low, a Level 1 concern is flagged or the case is escalated. The rationale becomes final only after the reviewer confirms "I have reviewed this rationale and take ownership of it". Every action is written to an audit trail (time, person, role, before and after, reason), and the consistency monitor shows where people and the AI diverge.

4. Personal Reflection

I came to this project from quantitative finance, where I model limit order book data and judge a model by its out-of-sample error. **Building Simrule changed what I think AI is for in a regulated industry: its main value is not deciding faster, but making each judgment explicit enough for a person to check.** The pilot showed this directly. The first prompt already pointed in the right direction on all 10 pilot rules, yet 9 of the 10 assessments failed validation because the reasoning and evidence were too thin to show a reviewer. **For a regulator, a correct answer that cannot be explained is close to worthless, so I now see transparency as the product rather than a feature.** Separating the findings from a deterministic scoring rule is what made the system auditable: the same findings always give the same recommendation, and a reviewer can point to the exact finding they disagree with.

I am a CEMS MIM student, currently on exchange in London. **I want to work in quantitative or AI roles in Japan, where I expect model governance to be at least as demanding as in this project. I plan to carry three habits from Simrule into that work: write the schema before the prompt, keep model judgment separate from the rule that acts on it, and test a model's explanations as rigorously as its accuracy. In trading research I would apply the same split, letting a model propose signals with evidence while transparent, versioned code decides how they are used, so**

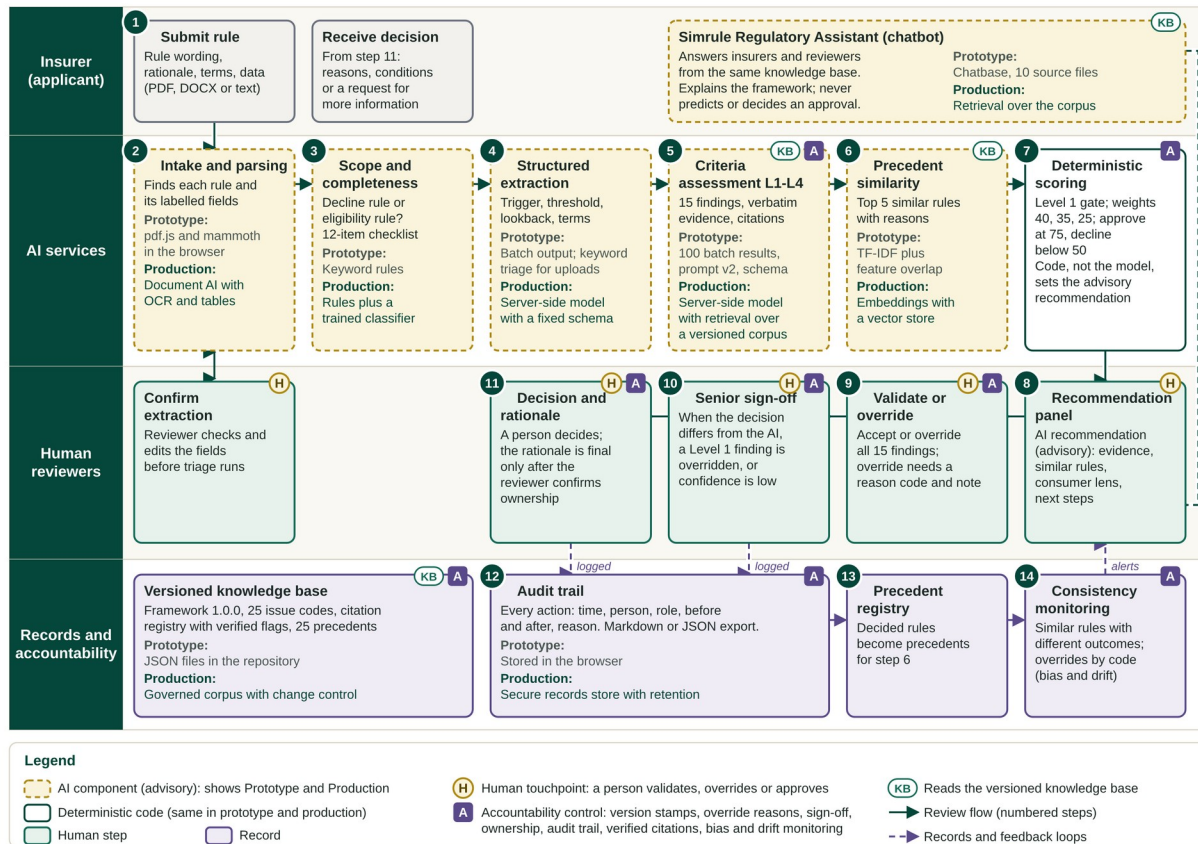
a risk committee can audit both. I also want to design the audit trail first rather than last, because it is the feature that makes everything else trustworthy.

What surprised me most was how often the checks I trusted were too weak. The automated consistency check passed 15 of 16 similar pairs with different outcomes, but reading them showed that 8 explanations described the similarity without ever saying why the outcomes differed. The end-to-end test, not the unit tests, found that PDF upload failed in Chromium because a library called a JavaScript method the browser does not yet support. I was also surprised that the hardest problems were not technical: deciding how severe a missing actuarial study should be, or when a postal code becomes a proxy, needed judgment I could only write down after seeing many cases. The actuarial severity rule had to be clarified at batch 7, when read literally it would have sent every novel administrative rule to an information request. The lesson I take away is that trust in AI is earned by making disagreement easy: a reviewer must see the evidence, be able to override it with a reason, and know that someone else will check.

Appendix: Architecture and AI-Use Disclosure

Simrule: AI-assisted review of auto insurance decline rules

Solution architecture. The AI advises; people decide; every step is recorded. Numbers follow the review flow.



Academic prototype for Ivey Online coursework. Not affiliated with or representative of the Financial Services Regulatory Authority of Ontario (FSRA). All insurer submissions and precedent records are fictional.

Figure 1. Simrule solution architecture. Numbered steps follow one submission through four swimlanes; H marks human touchpoints and A marks accountability controls. Full size: deliverables/architecture/simrule-architecture.pdf and the How it works page of the live app.

AI-use disclosure

Claude Code (Anthropic) was the engineering and analysis partner for this assignment. Working from my written brief, it wrote the application code and tests, generated the 100 structured assessments with a versioned prompt (the deterministic scoring code, not the model, produces each recommendation), produced the chatbot sources, avatar script and diagram, and drafted this report from the project logs (PROGRESS.md and prompt-log.md). Every number in the report comes from those logs. Legal references were not verified against official sources and are listed for checking in VERIFY.md. I reviewed the outputs, made the final design decisions, and rewrote the personal reflection in my own words.